



Module 04 – Extra Class

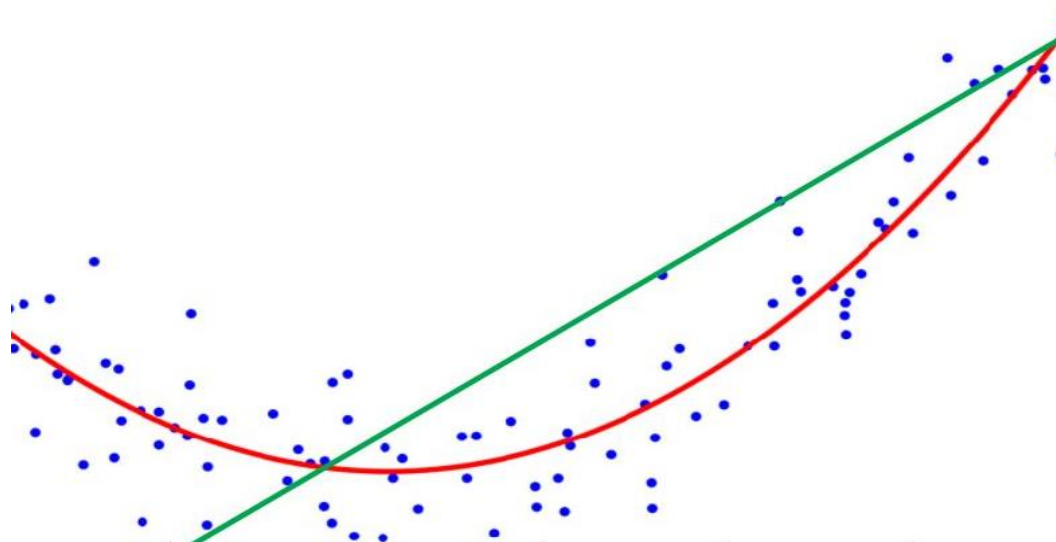
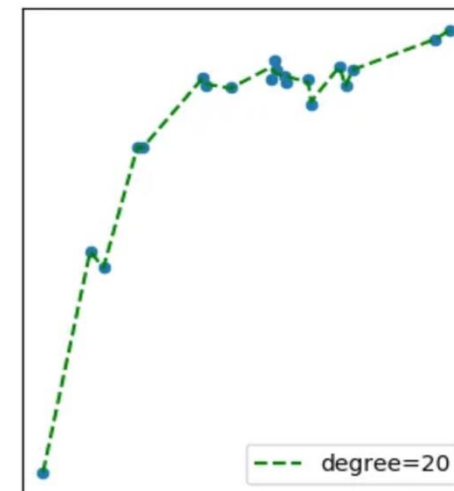
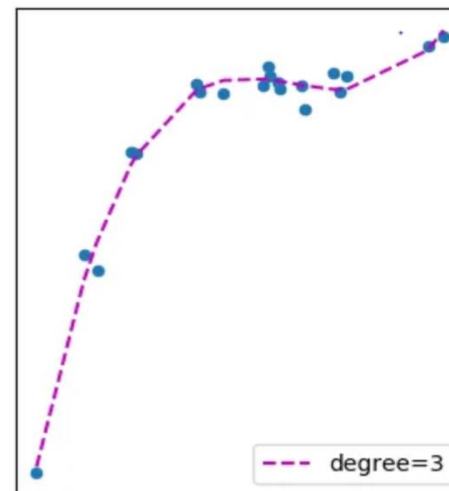
NON-LINEAR REGRESSION

Nguyen Quoc Thai

Objectives

Non-linear Regression

- ❖ Linear Regression
- ❖ Non-linear Regression
- ❖ Optimal Learning Rate
- ❖ Limitation of Linear Regression



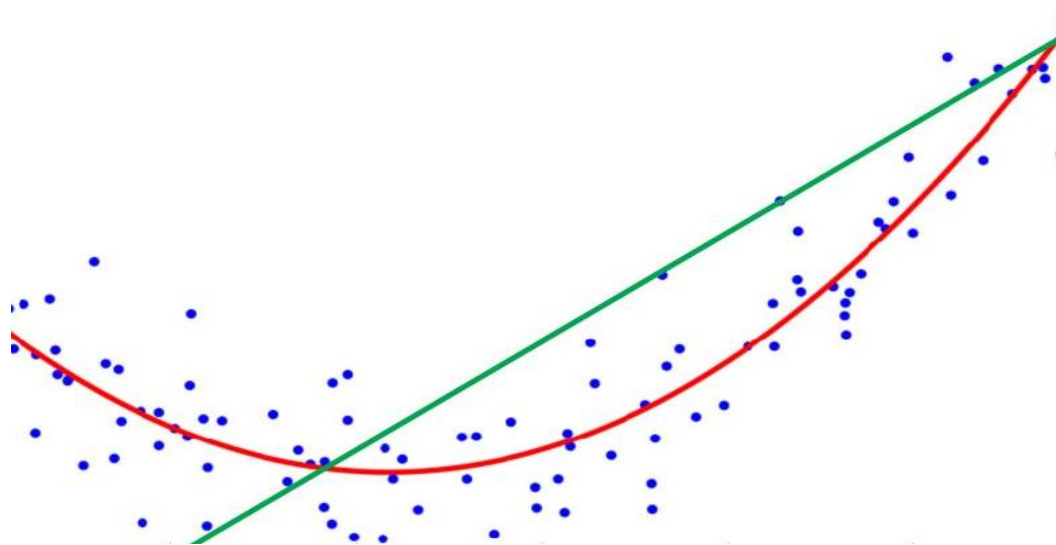
Polynomial Regression

- ❖ Moving Beyond Linearity
- ❖ Polynomial Feature
- ❖ Modeling
- ❖ Degree Choice
- ❖ Non-linear Regression for Fish Dataset

Outline

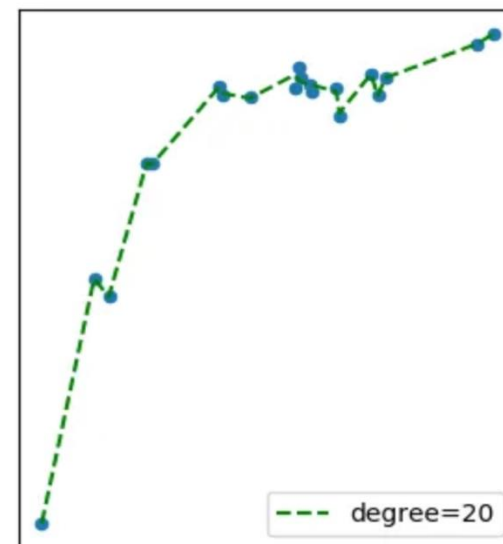
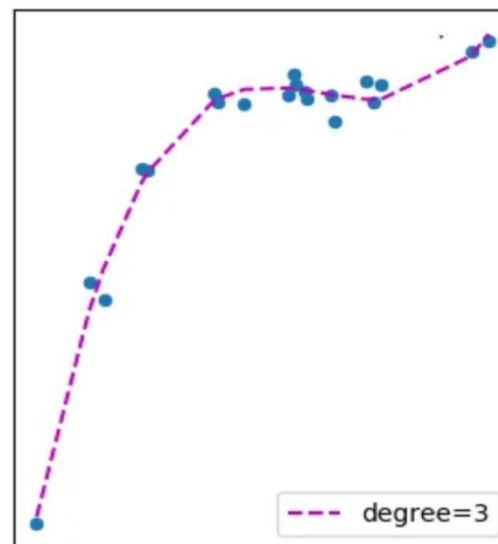
SECTION 1

Non-linear Regression



SECTION 2

Polynomial Regression



Non-linear Regression



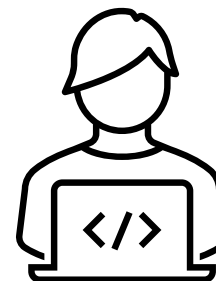
Introduction

Data

Level	Salary
0	8
1	15
2	18
3	22
4	26
5	30
6	38
7	47

Level	Salary
3,5	???
10	???

Learning



Prediction

Non-linear Regression

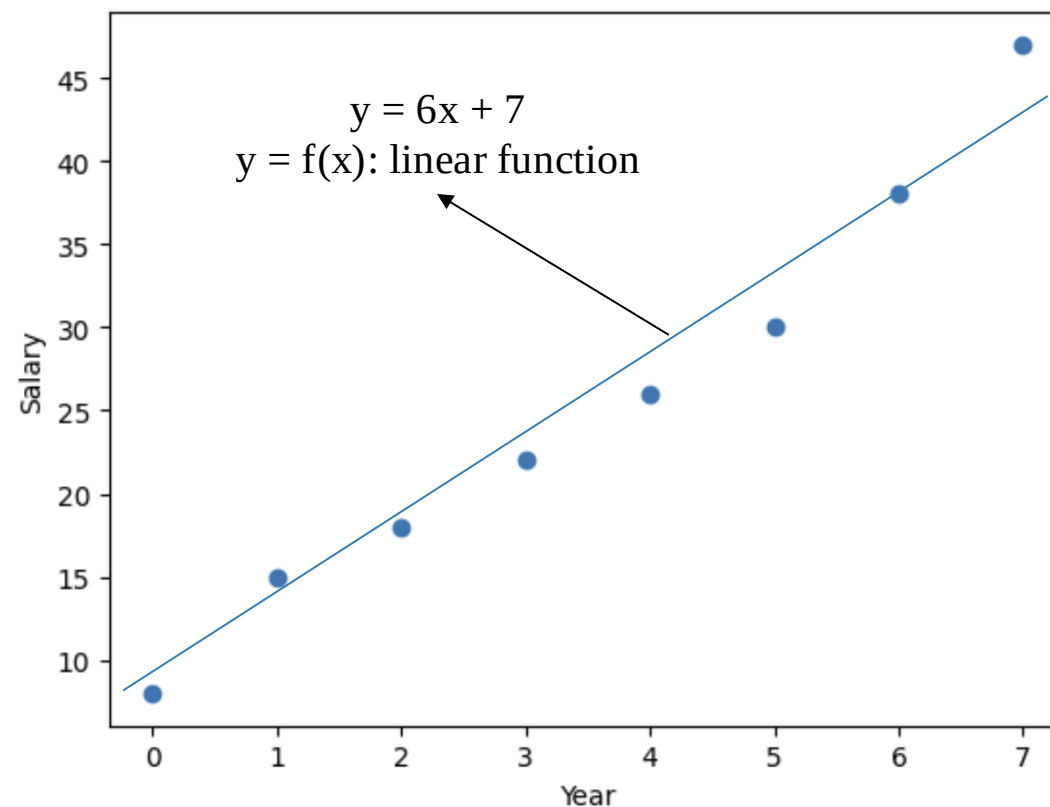


Linear Regression

Data

Level	Salary
0	8
1	15
2	18
3	22
4	26
5	30
6	38
7	47

Visualization



Non-linear Regression



Linear Regression

Data

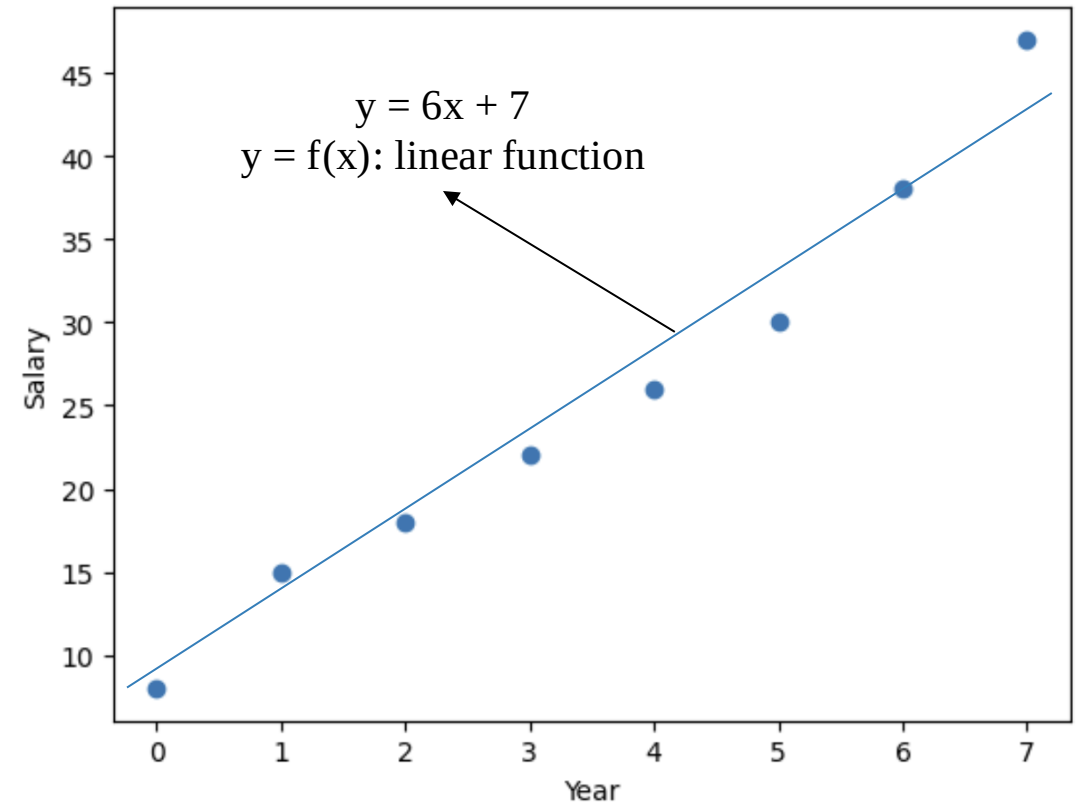
Level	Salary
0	8
1	15
2	18
3	22
4	26
5	30
6	38
7	47

Modeling

$$y = ax + b$$

Find a and b to fit the data

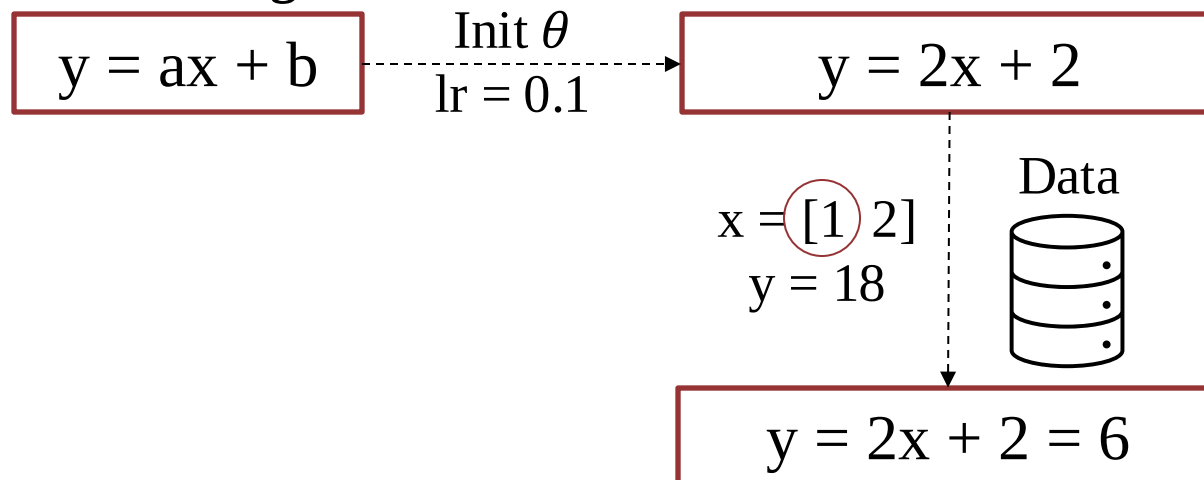
Visualization



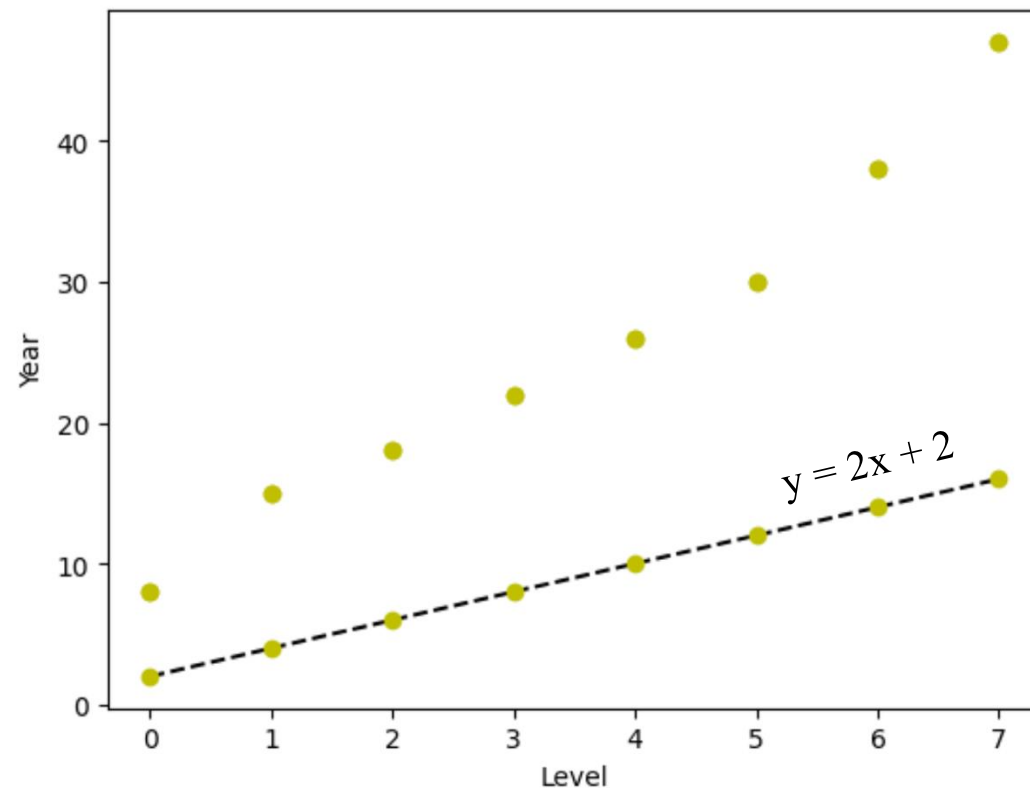
Non-linear Regression

! Linear Regression using Gradient Descent

Modeling



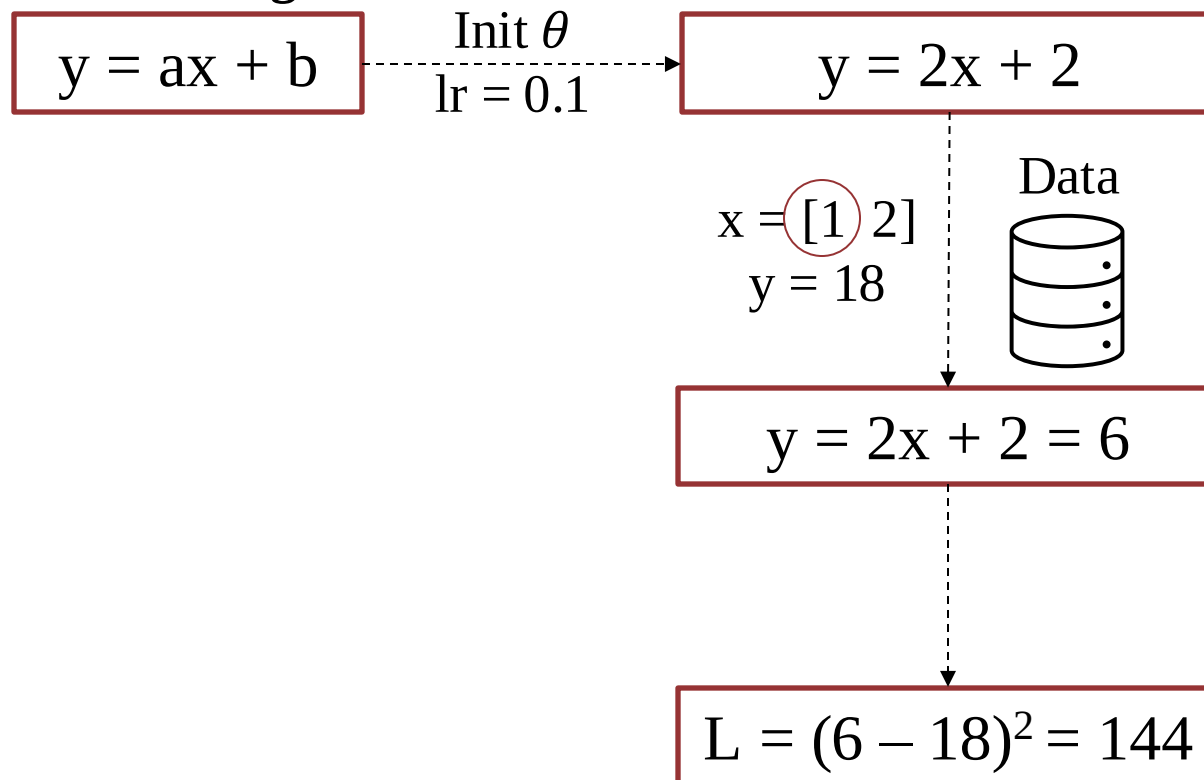
Visualization



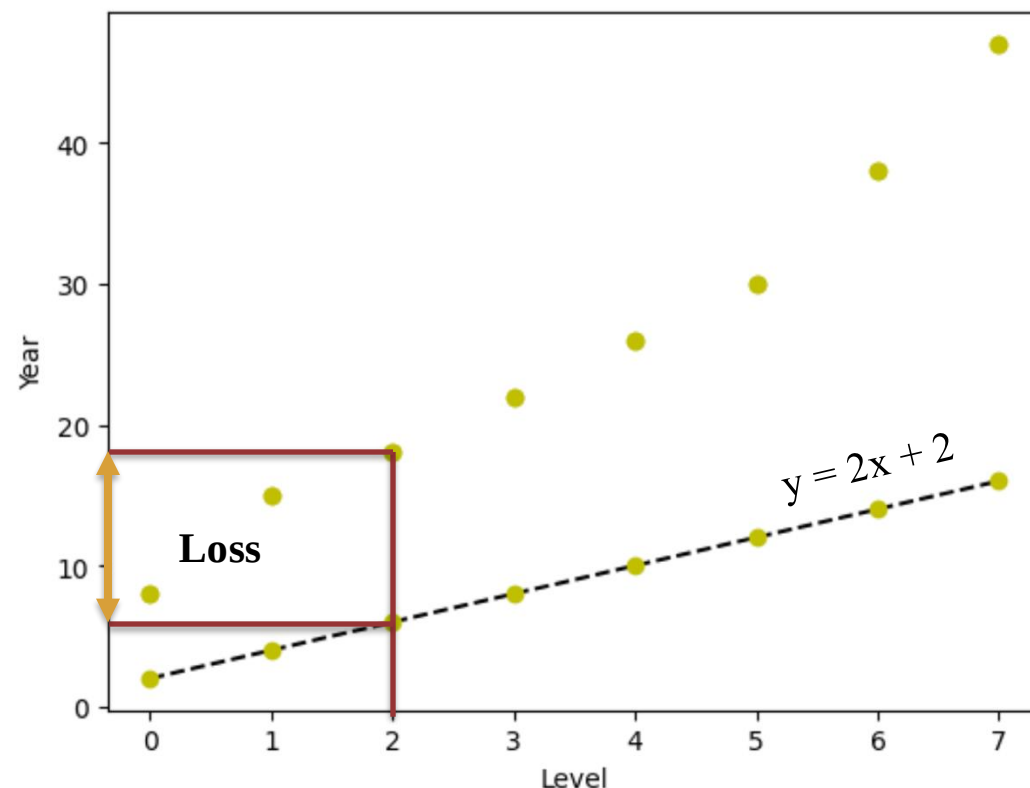
Non-linear Regression

! Linear Regression using Gradient Descent

Modeling



Visualization

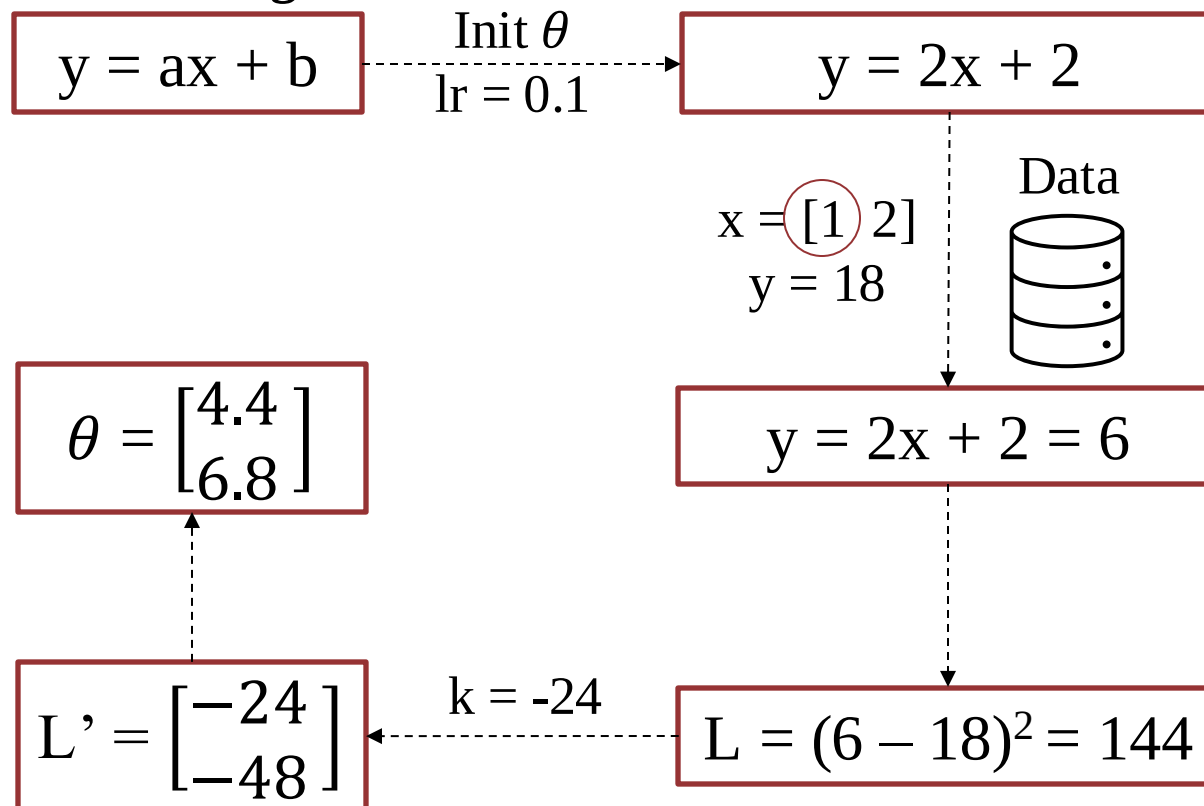


Difference between
predicted and actual value

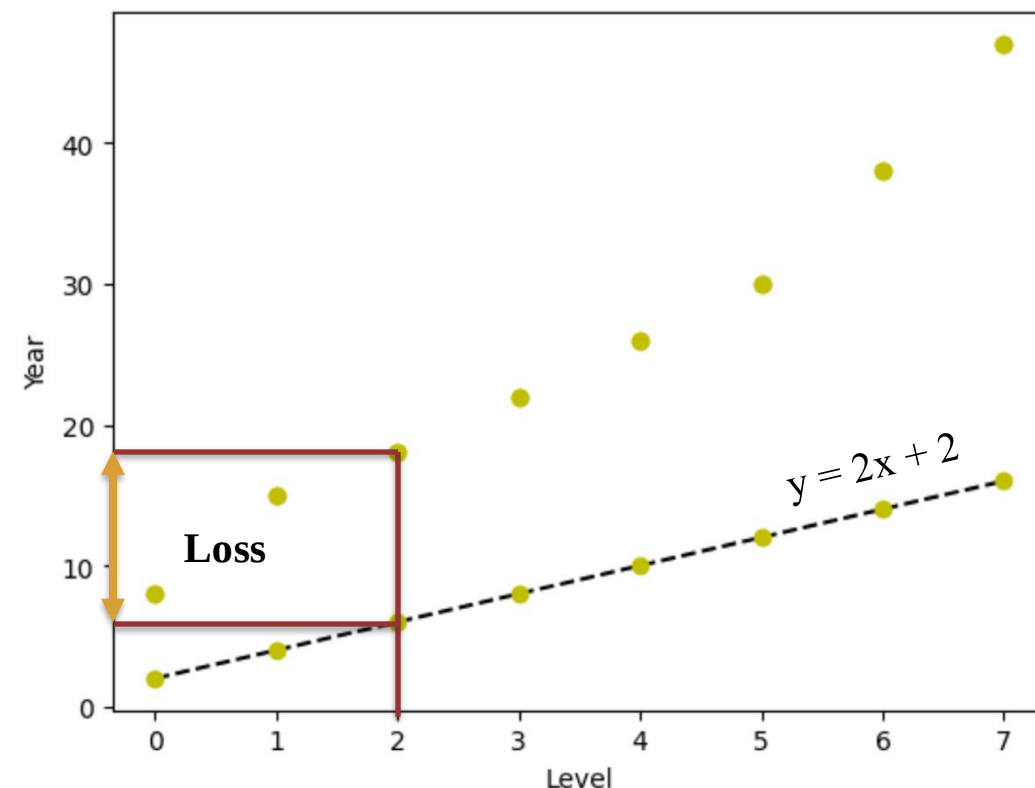
Non-linear Regression

! Linear Regression using Gradient Descent

Modeling



Visualization



Difference between
predicted and actual value

Non-linear Regression



Linear Regression using Gradient Descent

Modeling

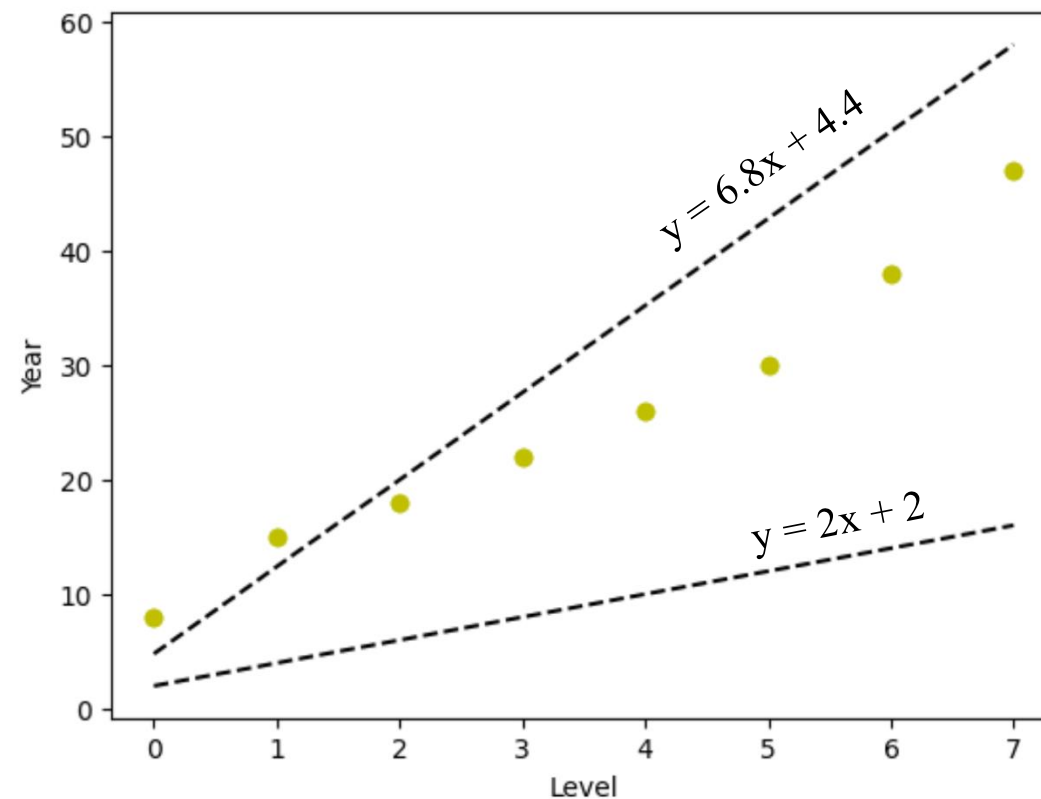
$$y = ax + b$$

$$y = 2x + 2$$

Updated

$$y = 6.8x + 4.4$$

Visualization



Non-linear Regression



Linear Regression using Gradient Descent

Data

Level	Salary
0	8
1	15
2	18
3	22
4	26
5	30
6	38
7	47

Inputs / Features

$$X = \begin{bmatrix} 1 & \varphi_1(1) & \dots & \varphi_{d-1}(1) \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \varphi_1(N) & \dots & \varphi_{d-1}(N) \end{bmatrix}$$

Target

$$Y = \begin{bmatrix} y(1) \\ \vdots \\ y(N) \end{bmatrix}$$

Weight

$$\theta = \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_{d-1} \end{bmatrix}$$

Predict

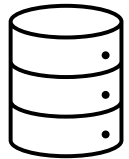
$$\hat{Y} = \begin{bmatrix} \theta_0 + \theta_1 * \varphi_1(1) + \dots + \theta_{d-1} \varphi_{d-1}(1) \\ \vdots \\ \theta_0 + \theta_1 * \varphi_1(N) + \dots + \theta_{d-1} \varphi_{d-1}(N) \end{bmatrix}$$

Non-linear Regression



Linear Regression using Gradient Descent

Data



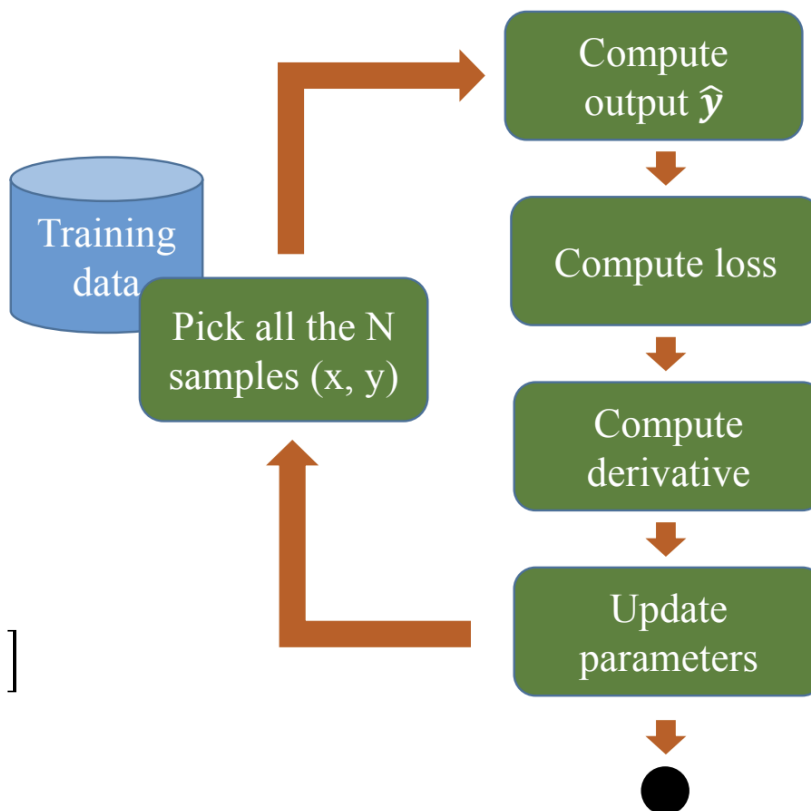
$$X = \begin{bmatrix} 1 & \varphi_1(1) & \dots & \varphi_{d-1}(1) \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \varphi_1(N) & \dots & \varphi_{d-1}(N) \end{bmatrix}$$

$$Y = \begin{bmatrix} y(1) \\ \vdots \\ y(N) \end{bmatrix}$$

$$y = ax + b \quad \theta = \begin{bmatrix} b \\ a \end{bmatrix}$$

$$\theta = \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_{d-1} \end{bmatrix}$$

Learning



1) Pick all the N samples from training data

2) Compute output $\hat{y}$

$$\hat{y} = x\theta$$

3) Compute loss

$$L(\hat{y}, y) = (\hat{y} - y) \odot (\hat{y} - y)$$

4) Compute derivative

$$k = 2(\hat{y} - y)$$

$$L'_{\theta} = x^T k$$

5) Update parameters

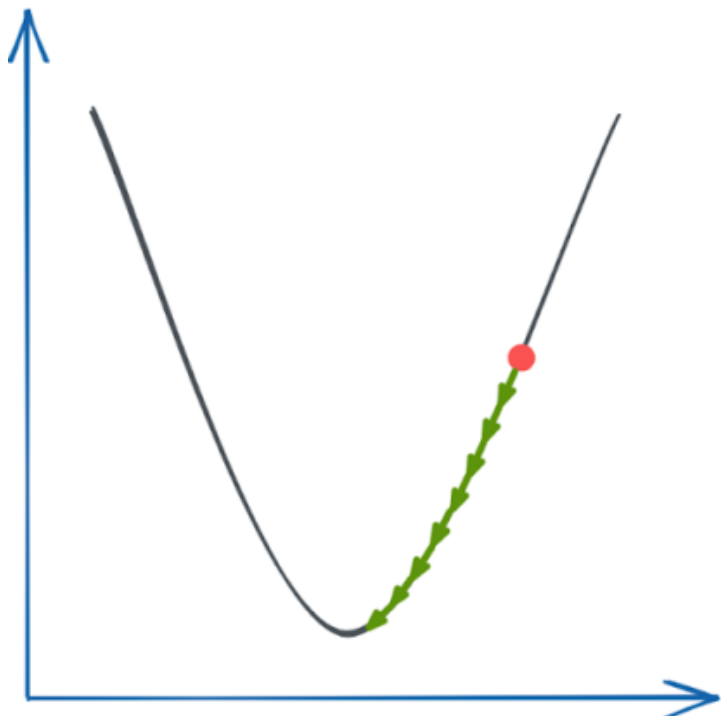
$$\theta = \theta - \eta \frac{L'_{\theta}}{N} \quad \eta \text{ is learning rate}$$

Non-linear Regression

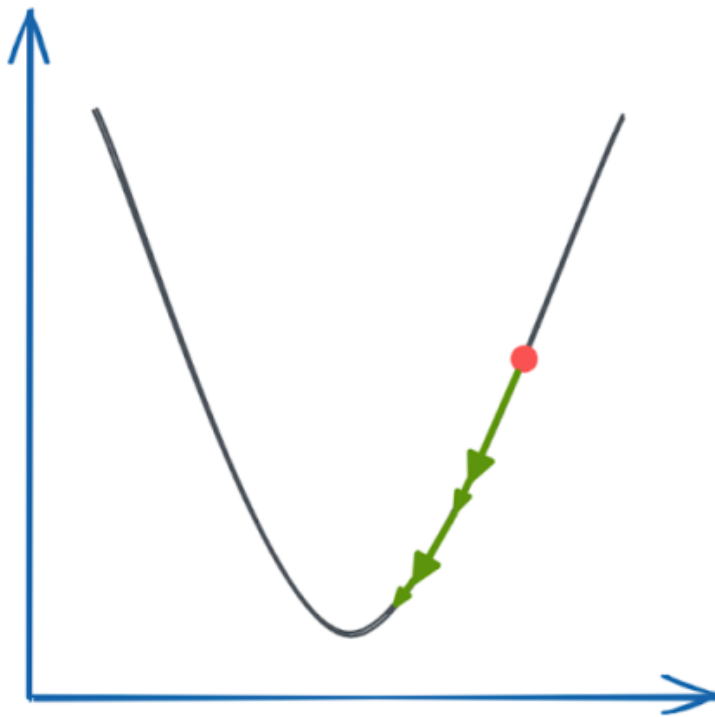


Optimal Learning Rate

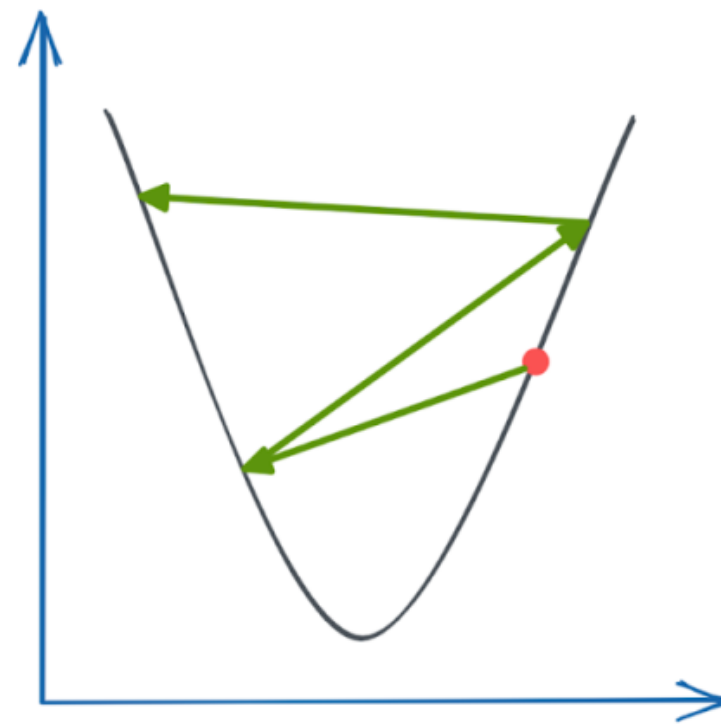
Slow



Optimal



High



Non-linear Regression



Limitation

Modeling

$$y = ax + b$$


The **main disadvantage** of this technique is that **the model is linear in both the parameters and the features**. This is a very restrictive assumption, quite of the often **data exhibits behaviours that are nonlinear in the features**

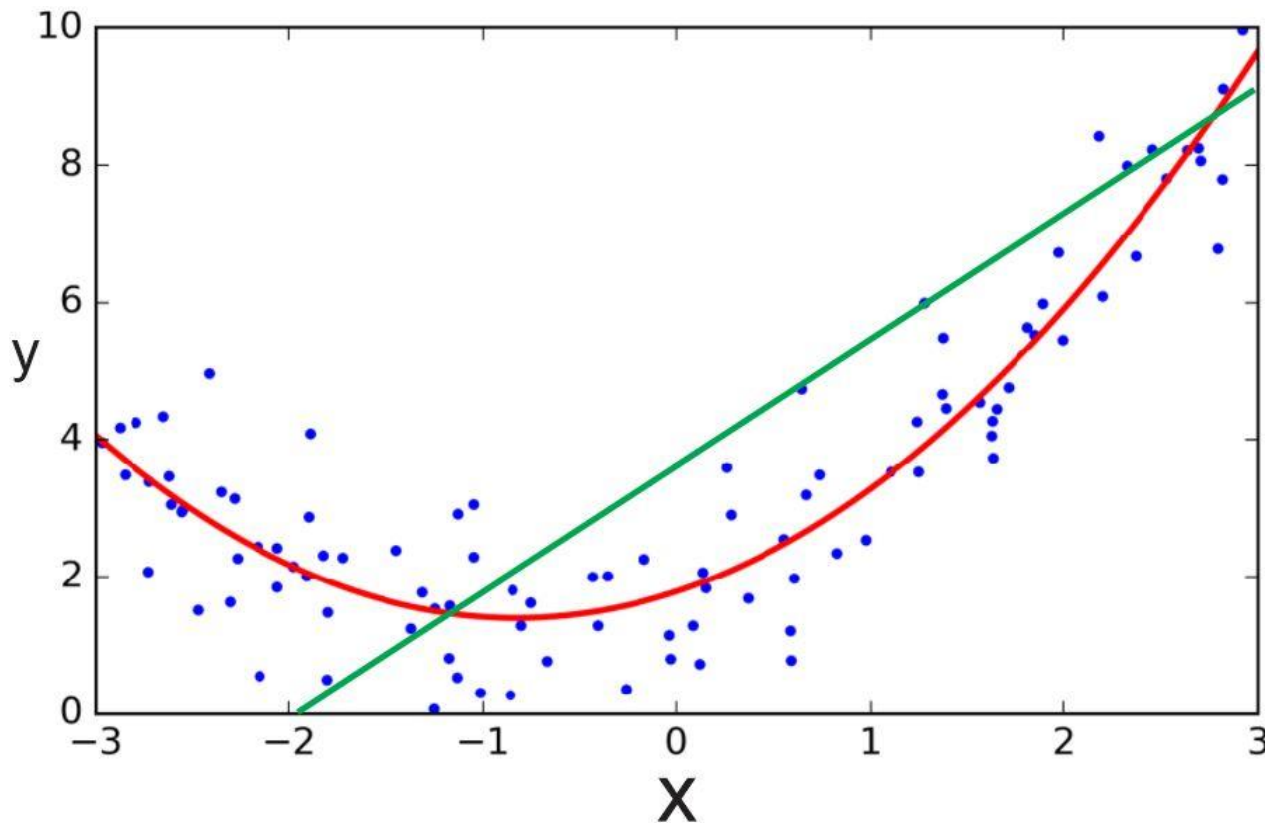
Extend this approach to more flexible models...

Non-linear Regression



Moving Beyond Linearity

Data Visualization



Linear function

$$\hat{y}(i) = \theta_0 + \theta_1 * \varphi(i)$$

Polynomial function

$$\hat{y}(i) = \theta_0 + \theta_1 * \varphi(i) + \theta_2 * \varphi(i)^2$$

Nonlinear regression estimates the output based on nonlinear function

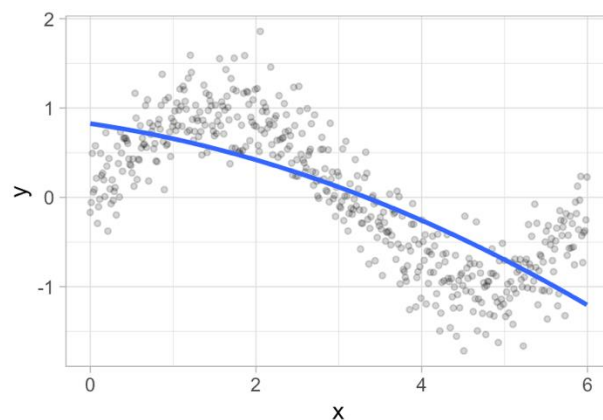
Notice that the prediction is **still linear in the parameters** but **nonlinear in the features**

Non-linear Regression

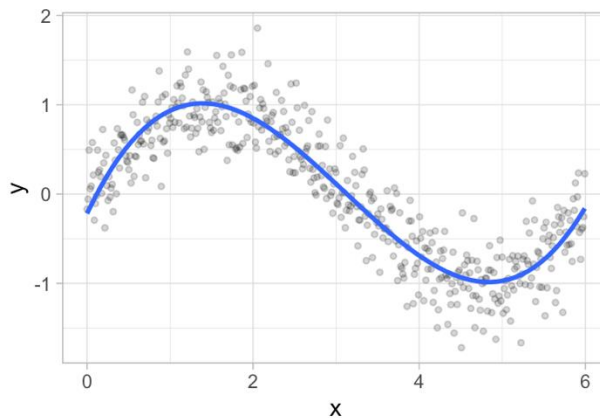


Moving Beyond Linearity

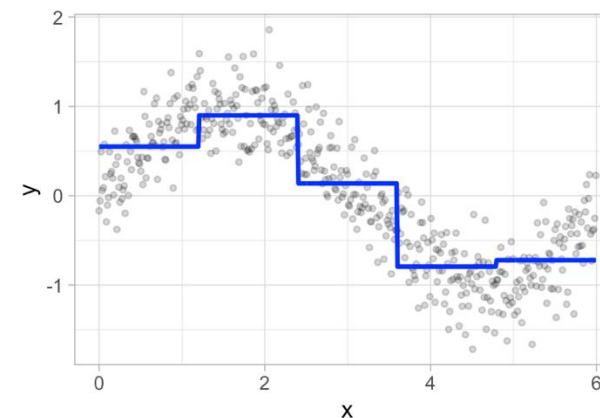
2-degree polynomial function



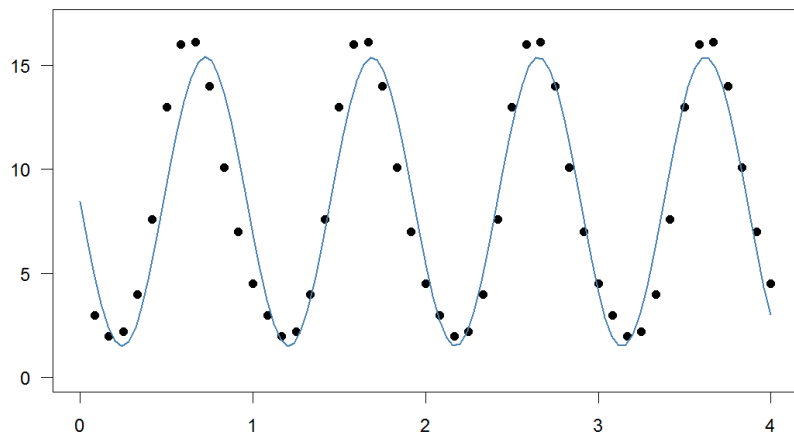
3-degree polynomial function



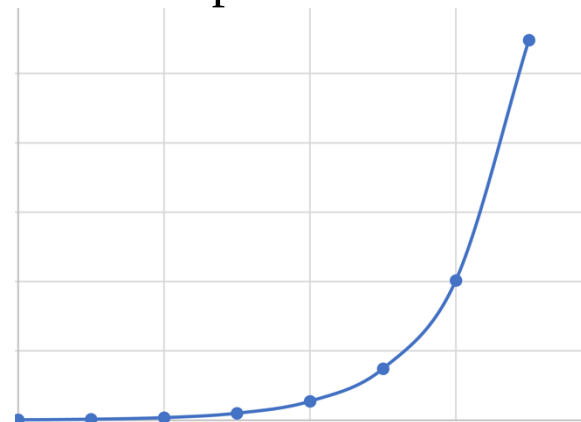
Step function



Sinusoidal



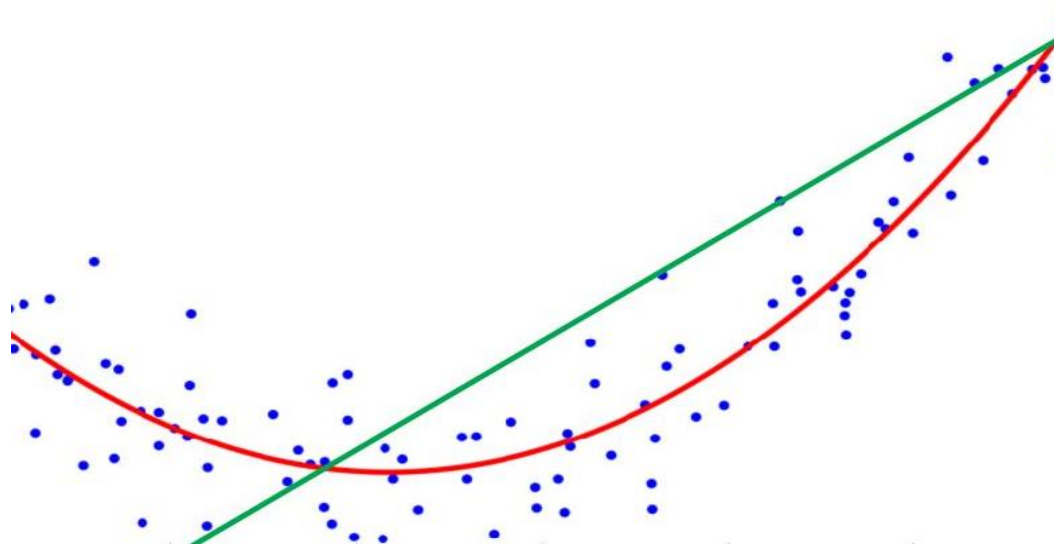
Exponential function



Outline

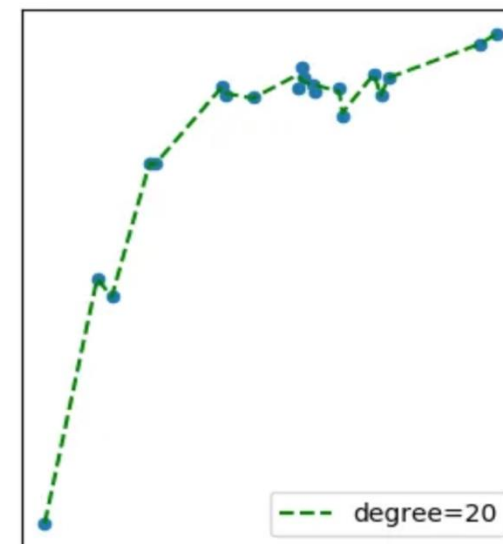
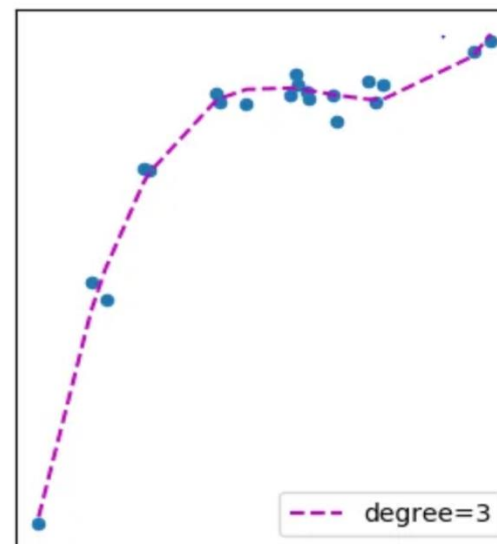
SECTION 1

Non-linear Regression



SECTION 2

Polynomial Regression



Polynomial Regression



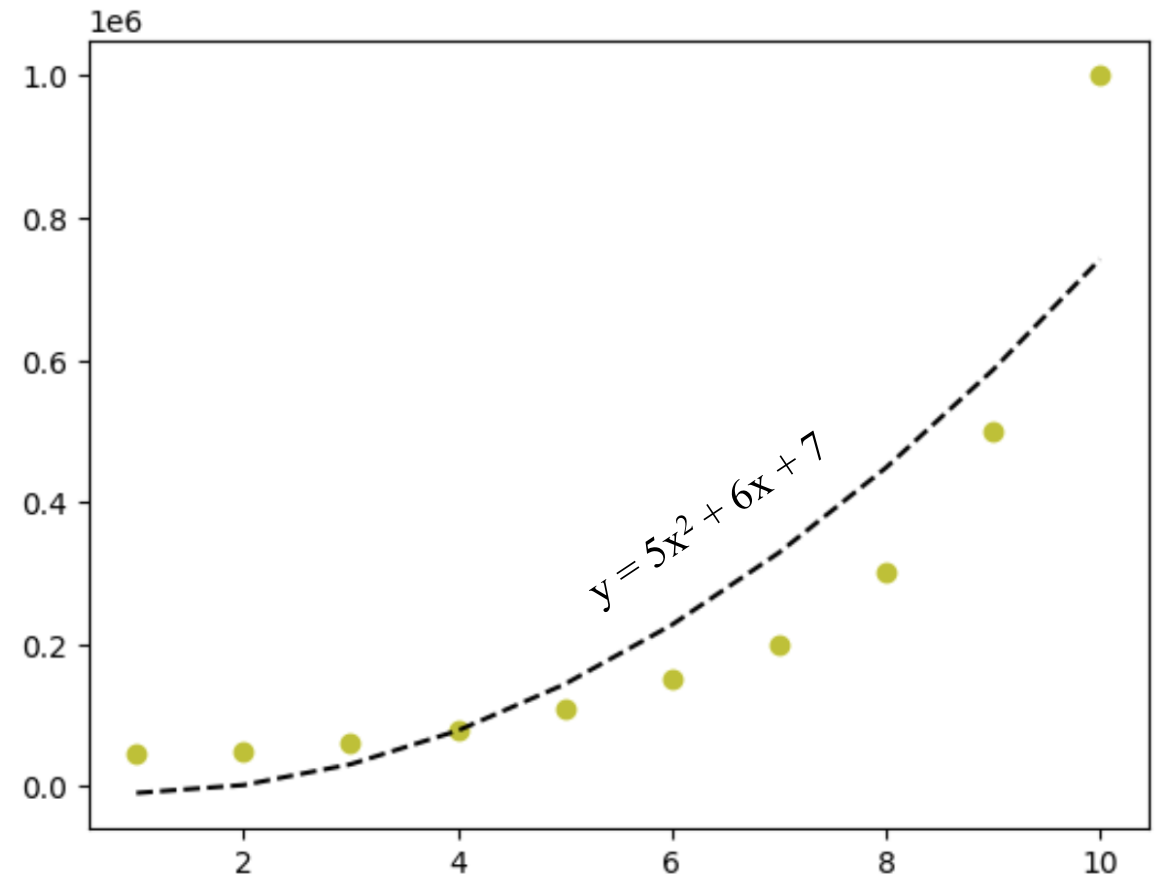
Polynomial Features

2-degree polynomial function

$$\hat{y}(i) = \theta_0 * 1 + \theta_1 * \varphi(i) + \theta_2 * \varphi(i)^2$$



Find θ_0 , θ_1 , θ_2 to fit the data



Polynomial Regression



Polynomial Features

2-degree polynomial function

$$\hat{y}(i) = \theta_0 * 1 + \theta_1 * \varphi(i) + \theta_2 * \varphi(i)^2$$

Create polynomial feature



$\psi(\cdot)$ is referred to as basis function and it can be seen as a function that transforms the input in some way (In this case its powers function)

Polynomial Regression

! Polynomial Features

2-degree polynomial function

$$\hat{y}(i) = \theta_0 * 1 + \theta_1 * \varphi(i) + \theta_2 * \varphi(i)^2$$

Create polynomial feature

Data

Level	Salary
0	45000
1	50000
2	60000
3	80000
4	110000
5	160000

Input

0
1
2
3
4
5

1

1
1
1
1
1
1

$\psi(\varphi(i))$

$\varphi(i)$

0
1
2
3
4
5

$\varphi(i)^2$

0
1
4
9
16
25

Polynomial Regression



Polynomial Features

2-degree polynomial function

$$\hat{y}(i) = \theta_0 * 1 + \theta_1 * \varphi(i) + \theta_2 * \varphi(i)^2$$

Features			Target
1	$\varphi(i)$	$\varphi(i)^2$	
1	0	0	45000
1	1	1	50000
1	2	4	60000
1	3	9	80000
1	4	16	110000
1	5	25	160000

Polynomial Regression



Polynomial Features

3-degree polynomial function

$$\hat{y}(i) = \theta_0 * 1 + \theta_1 * \varphi(i) + \theta_2 * \varphi(i)^2 + \theta_3 * \varphi(i)^3$$

Features				Target
1	$\varphi(i)$	$\varphi(i)^2$	$\varphi(i)^3$	
1	0	0	0	45000
1	1	1	1	50000
1	2	4	8	60000
1	3	9	27	80000
1	4	16	64	110000
1	5	25	125	160000

Polynomial Regression



Polynomial Features

Input

0
1
2
3
4
5

Features

1	$\varphi(i)$	$\varphi(i)^2$
1	0	0
1	1	1
1	2	4
1	3	9
1	4	16
1	5	25

Algorithm

```
def create_polynomial_features(X, degree=2):  
    """Creates the polynomial features  
    Args:  
        X: A torch tensor for the data.  
        degree: A integer for the degree of  
              the generated polynomial function.  
    """  
    X_new = X  
    for d in range(2, degree+1):  
        X_new = np.c_[X_new, np.power(X, d)]  
    return X_new
```

Polynomial Regression



Polynomial Features

Input

0
1
2
3
4
5

Features

1	$\varphi(i)$	$\varphi(i)^2$
1	0	0
1	1	1
1	2	4
1	3	9
1	4	16
1	5	25

```
1 from sklearn.preprocessing import PolynomialFeatures
```

```
1 poly_features = PolynomialFeatures(degree=2)
```

```
1 X.to_frame()
```

(10, 1)

```
1 X_poly = poly_features.fit_transform(X.to_frame())
```

```
2 X_poly
```

```
array([[ 1.,   1.,   1.],
       [ 1.,   2.,   4.],
       [ 1.,   3.,   9.],
       [ 1.,   4.,  16.],
       [ 1.,   5.,  25.],
       [ 1.,   6.,  36.],
       [ 1.,   7.,  49.],
       [ 1.,   8.,  64.],
       [ 1.,   9.,  81.],
       [ 1.,  10., 100.]])
```


Polynomial Regression



Model

Data

Level	Salary
0	45000
1	50000
2	60000
3	80000
4	110000
5	160000

Inputs / Features with b-degree

$$X = \begin{bmatrix} 1 & \varphi_1(1) & \dots & \varphi_1(1)^b \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \varphi_1(N) & \dots & \varphi_1(N)^b \end{bmatrix}$$

Target

$$Y = \begin{bmatrix} y(1) \\ \vdots \\ y(N) \end{bmatrix}$$

Weight

$$\theta = \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_b \end{bmatrix}$$

Predict

$$\hat{Y} = \begin{bmatrix} \theta_0 + \theta_1 * \varphi_1(1) + \dots + \theta_b \varphi_1(1)^b \\ \vdots \\ \theta_0 + \theta_1 * \varphi_1(N) + \dots + \theta_b \varphi_1(N)^b \end{bmatrix}$$

Polynomial Regression



Model

Nonlinear Regression Model

$$X = \begin{bmatrix} 1 & \varphi_1(1) & \dots & \varphi_1(1)^b \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \varphi_1(N) & \dots & \varphi_1(N)^b \end{bmatrix}$$

$$Y = \begin{bmatrix} y(1) \\ \vdots \\ y(N) \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_b \end{bmatrix}$$

$$\hat{Y} = \begin{bmatrix} \theta_0 + \theta_1 * \varphi_1(1) + \dots + \theta_b \varphi_1(1)^b \\ \vdots \\ \theta_0 + \theta_1 * \varphi_1(N) + \dots + \theta_b \varphi_1(N)^b \end{bmatrix}$$

Both models are linear
in the parameters

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N (\hat{y} - y)^2$$

Using Gradient Decent

Linear Regression Model

$$X = \begin{bmatrix} 1 & \varphi_1(1) \\ \vdots & \vdots \\ 1 & \varphi_1(N) \end{bmatrix}$$

$$Y = \begin{bmatrix} y(1) \\ \vdots \\ y(N) \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$$

$$\hat{Y} = \begin{bmatrix} \theta_0 + \theta_1 * \varphi_1(1) \\ \vdots \\ \theta_0 + \theta_1 * \varphi_1(N) \end{bmatrix}$$

Polynomial Regression

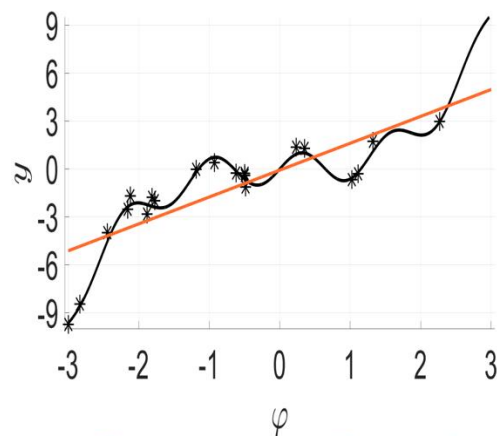


Degree Choice

b-degree polynomial function

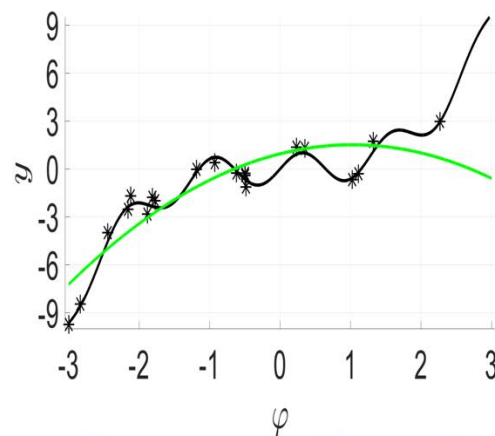
$$\hat{y}(i) = \theta_0 * 1 + \theta_1 * \varphi(i) + \theta_2 * \varphi(i)^2 + \dots + \theta_b * \varphi(i)^b$$

The choice of the degree of the polynomial is critical and depends on the dataset at hand

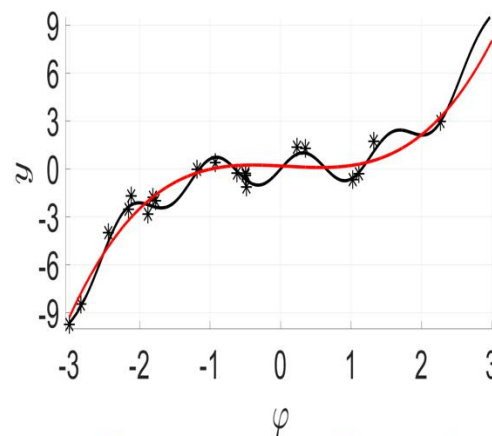


1-degree

Too simple/not flexible enough

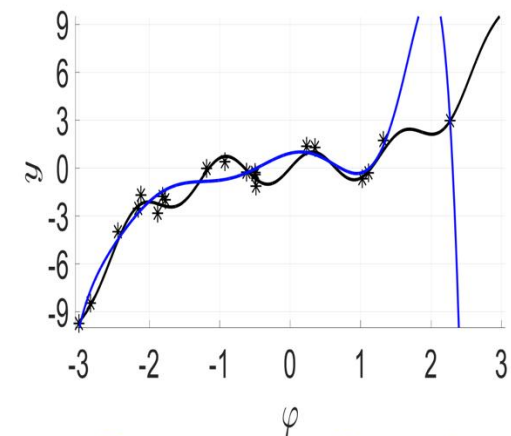


2-degree



3-degree

Just right



9-degree

Overfitting

Polynomial Regression



Degree Choice

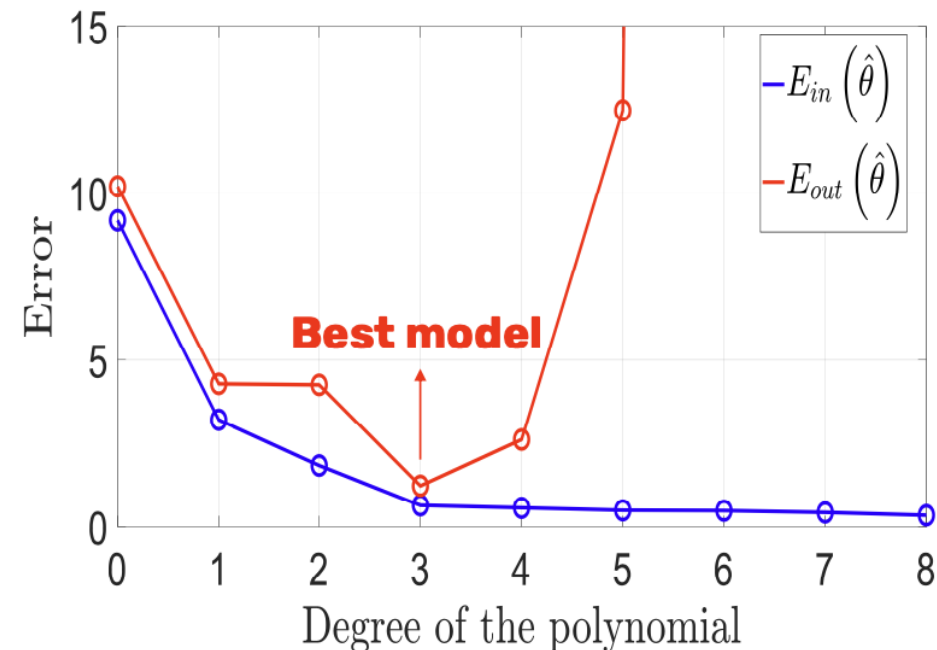
b-degree polynomial function

$$\hat{y}(i) = \theta_0 * 1 + \theta_1 * \varphi(i) + \theta_2 * \varphi(i)^2 + \dots + \theta_b * \varphi(i)^b$$

The choice of the degree of the polynomial is critical and depends on the dataset at hand

Good method for choice of the degree:
K-fold cross-validation

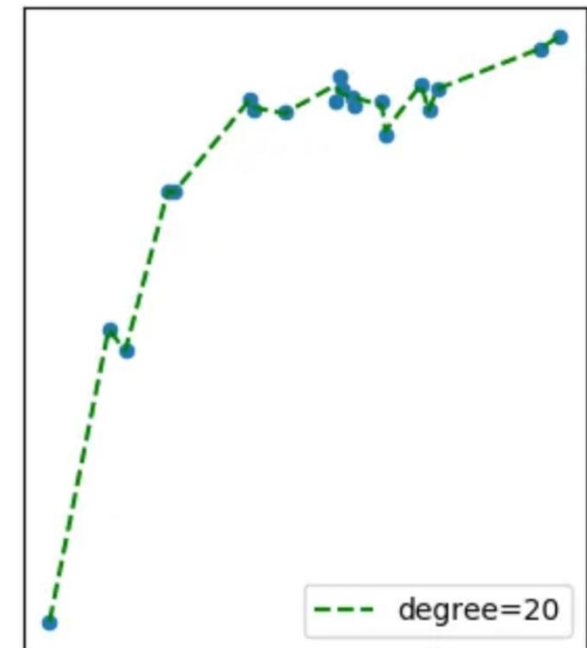
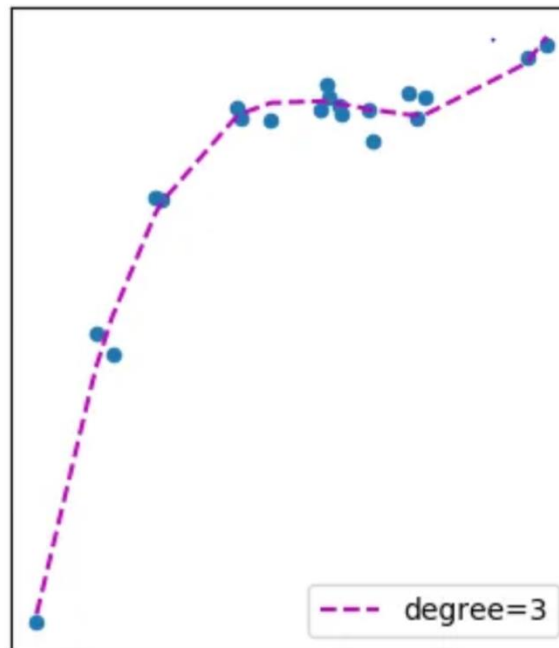
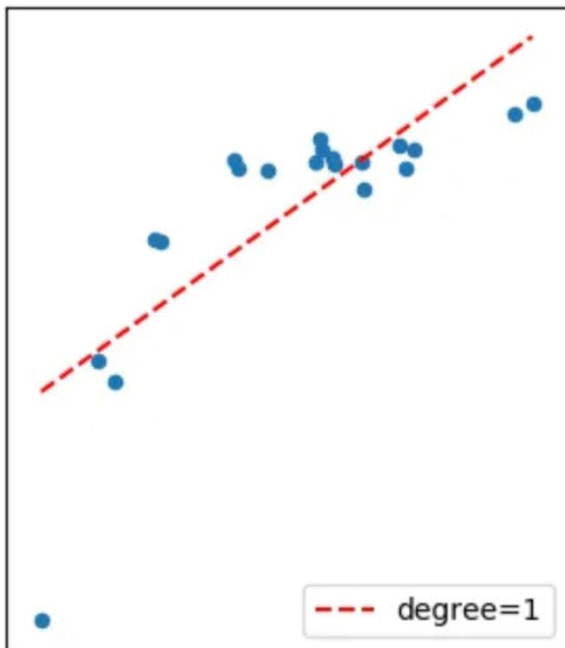
Choose the degree which has the
lowest out-of-sample error



Polynomial Regression

! Disadvantages

Increasing the degree of the polynomial always results in a model that is more sensitive to stochastic noise (even if that degree is the best one obtained from validation), especially at the boundaries (where we often have less data).



Polynomial Regression



Extension to The Multivariable Case

Build a machine learning model to predict the weight of the fish based on the body measurement data of seven types of fish species

	Species	Weight	VerticalLen	DiagonalLen	CrossLen	Height	Width
0	Bream	242.0	23.2	25.4	30.0	11.5200	4.0200
1	Bream	290.0	24.0	26.3	31.2	12.4800	4.3056
2	Bream	340.0	23.9	26.5	31.1	12.3778	4.6961
3	Bream	363.0	26.3	29.0	33.5	12.7300	4.4555
4	Bream	430.0	26.5	29.0	34.0	12.4440	5.1340

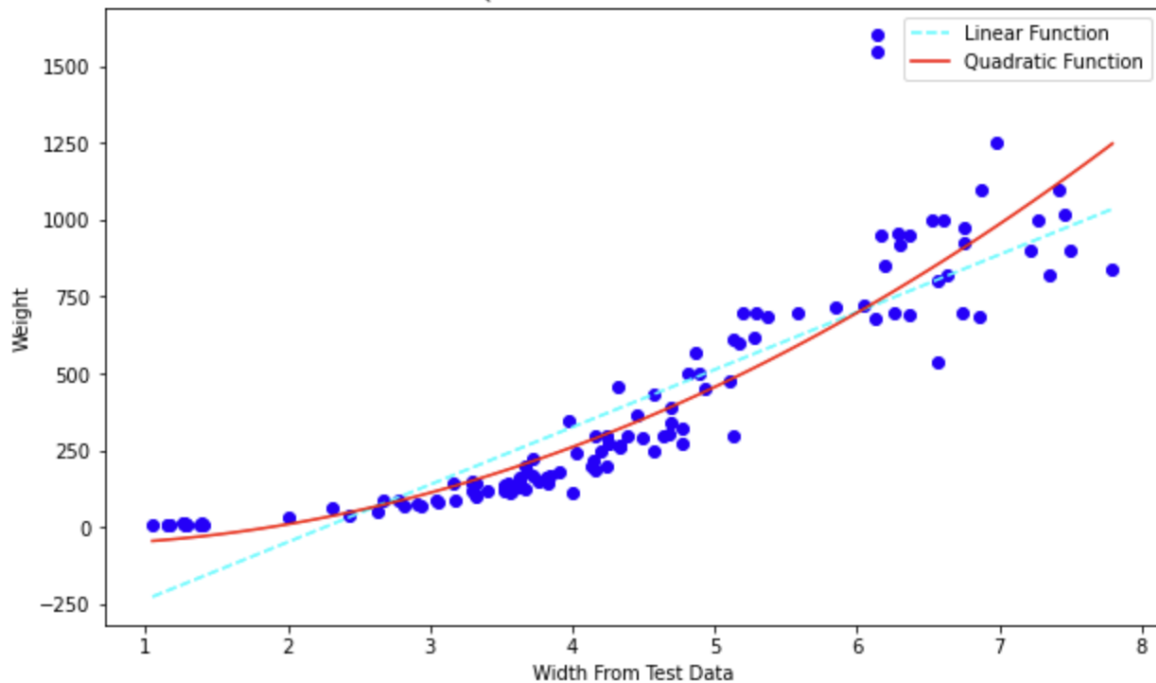
Polynomial Regression



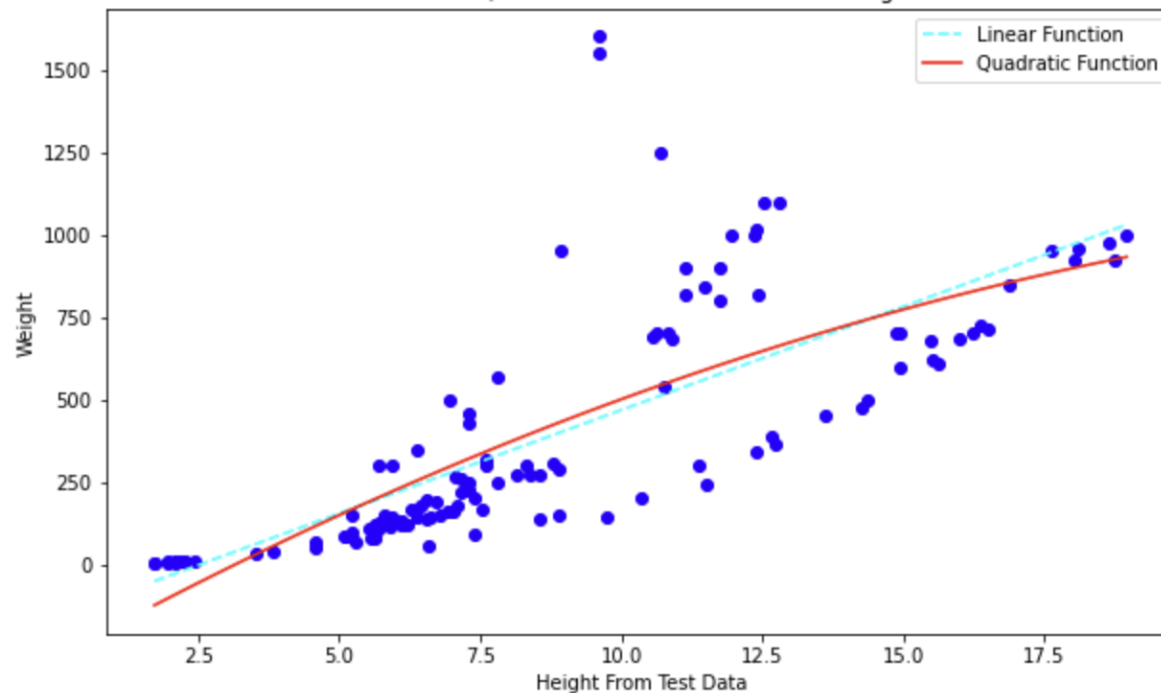
Extension to The Multivariable Case

Data Visualization

Linear Vs Quadratic Function For Feature Width



Linear Vs Quadratic Function For Feature Height



Polynomial Regression



Simple Approach

$$(a+b)^2 \Rightarrow a^2 + b^2 + a + b + 1$$

Multivariable

X	
0	2
1	1
2	2
3	1
4	2
5	1

1	$\varphi(i)$		$\varphi(i)^2$	
1	0	2	0	4
1	1	1	1	1
1	2	2	4	4
1	3	1	9	1
1	4	2	16	4
1	5	1	25	1

Polynomial Regression



Simple Approach

$$(a+b)^2 \Rightarrow a^2 + b^2 + a + b + 1$$

```
▶ def create_polynomial_features(X, degree=2):  
    """Creates the polynomial features  
    Args:  
        X: A torch tensor for the data.  
        degree: A integer for the degree of  
        the generated polynomial function.  
    """  
    X_mem = []  
    for X_sub in X.T:  
        X_sub = X_sub.T  
        X_new = X_sub  
        for d in range(2, degree+1):  
            X_new = np.c_[X_new, np.power(X_sub, d)]  
        X_mem.extend(X_new.T)  
    return np.c_[X_mem].T
```

Polynomial Regression



Advanced Approach

$$(a+b)^2 \Rightarrow a^2 + b^2 + ab + a + b + 1$$

Multivariable

X	
0	2
1	1
2	2
3	1
4	2
5	1

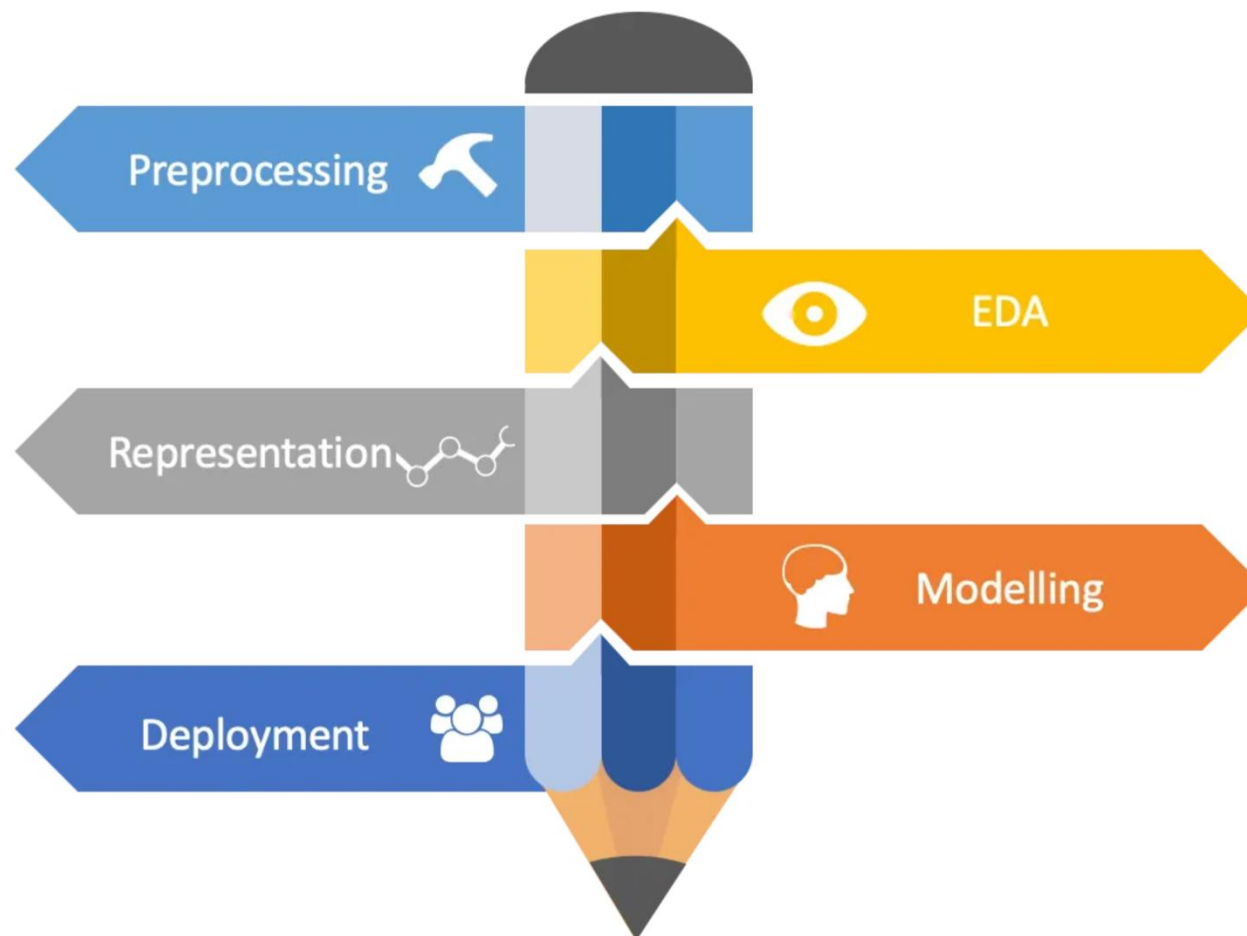
1	$\varphi(i)$		$\varphi(i)^2$		
1	0	2	0	4	0
1	1	1	1	1	1
1	2	2	4	4	4
1	3	1	9	1	3
1	4	2	16	4	8
1	5	1	25	1	5

QUIZ TIME

Polynomial Regression



Non-linear Regression for Fish Dataset



Polynomial Regression



Non-linear Regression for Fish Dataset

(1) - Preprocessing

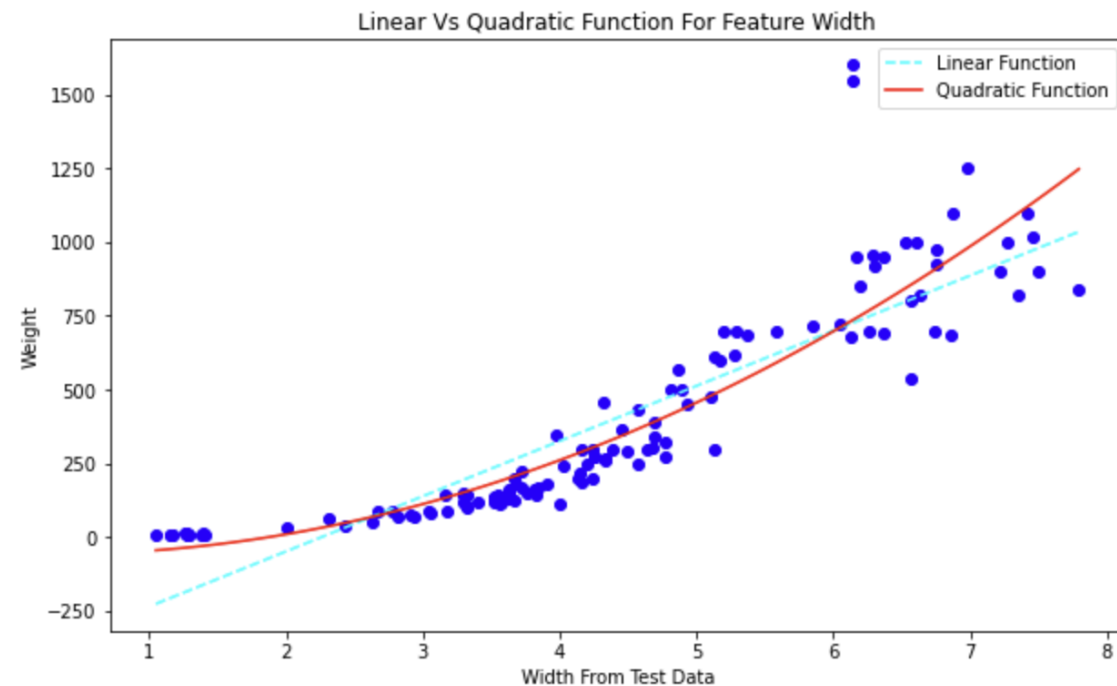
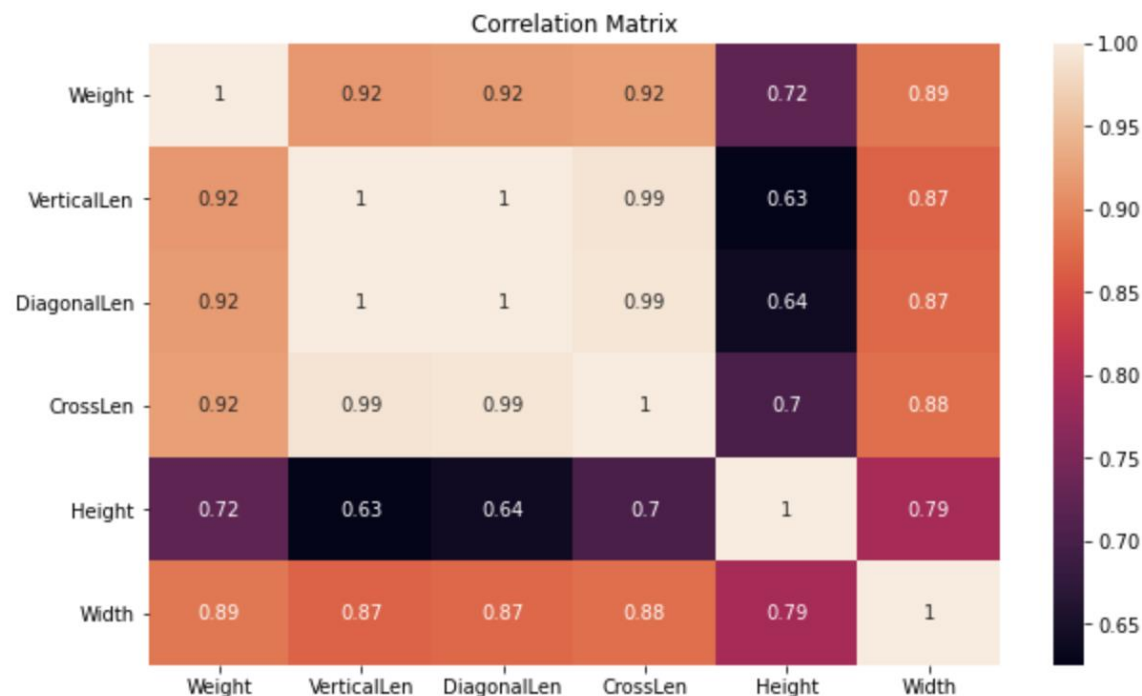
	Species	Weight	VerticalLen	DiagonalLen	CrossLen	Height	Width
0	Bream	242.0	23.2	25.4	30.0	11.5200	4.0200
1	Bream	290.0	24.0	26.3	31.2	12.4800	4.3056
2	Bream	340.0	23.9	26.5	31.1	12.3778	4.6961
3	Bream	363.0	26.3	29.0	33.5	12.7300	4.4555
4	Bream	430.0	26.5	29.0	34.0	12.4440	5.1340

Polynomial Regression



Non-linear Regression for Fish Dataset

(2) – EDA (Exploratory Data Analysis)



Polynomial Regression



Non-linear Regression for Fish Dataset

(3) – Representation + Polynomial Feature

Category Feature

	Species	Weight	VerticalLen	DiagonalLen	CrossLen	Height	Width
0	Bream	242.0	23.2	25.4	30.0	11.5200	4.0200
1	Bream	290.0	24.0	26.3	31.2	12.4800	4.3056
2	Bream	340.0	23.9	26.5	31.1	12.3778	4.6961
3	Bream	363.0	26.3	29.0	33.5	12.7300	4.4555
4	Bream	430.0	26.5	29.0	34.0	12.4440	5.1340

Polynomial Regression



Non-linear Regression for Fish Dataset

(3) – Representation + Polynomial Feature

Category Feature

	Species	Weight
0	Bream	242.0
1	Bream	290.0
2	Bream	340.0
3	Bream	363.0
4	Bream	430.0

Category Encoding

Index	Category
0	Bream
1	Parkki
2	Perch

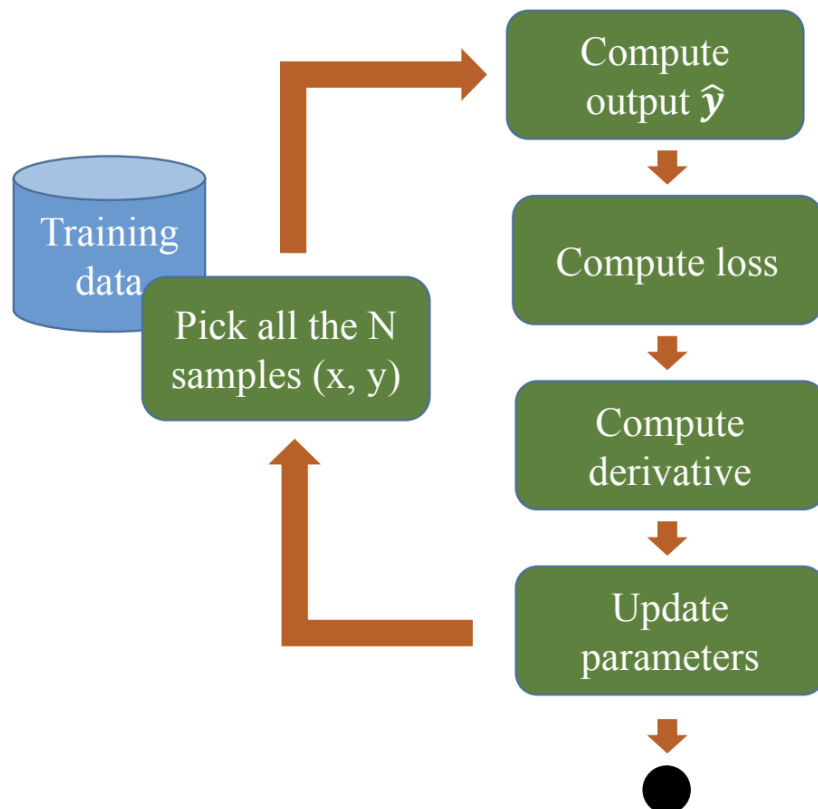
One Hot Encoding

Category			
Bream	[1	0	0]
Parkki	[0	1	0]
Perch	[0	0	1]

Polynomial Regression

! Non-linear Regression for Fish Dataset

(4) - Modeling



1) Pick all the N samples from training data

2) Compute output $\hat{y}$

$$\hat{y} = x\theta$$

3) Compute loss

$$L(\hat{y}, y) = (\hat{y} - y) \odot (\hat{y} - y)$$

4) Compute derivative

$$k = 2(\hat{y} - y)$$

$$L'_{\theta} = x^T k$$

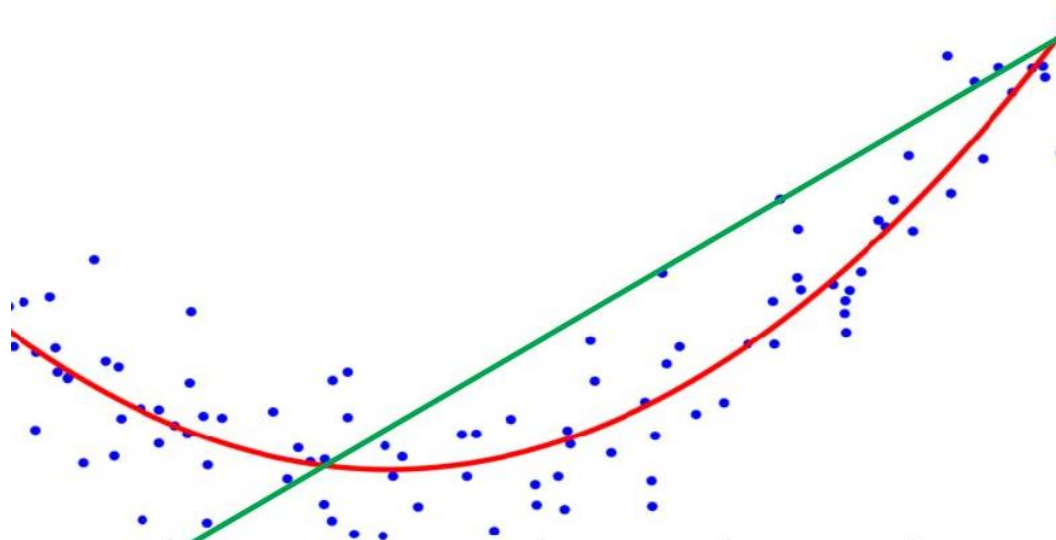
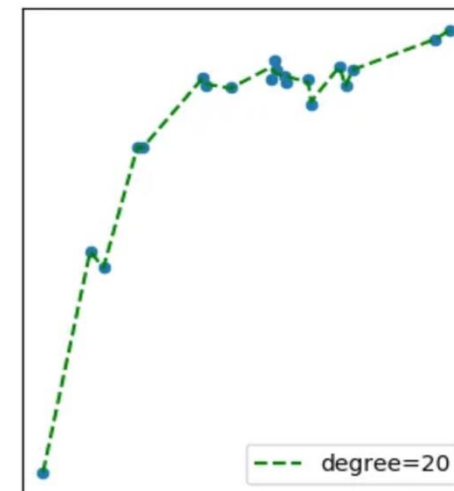
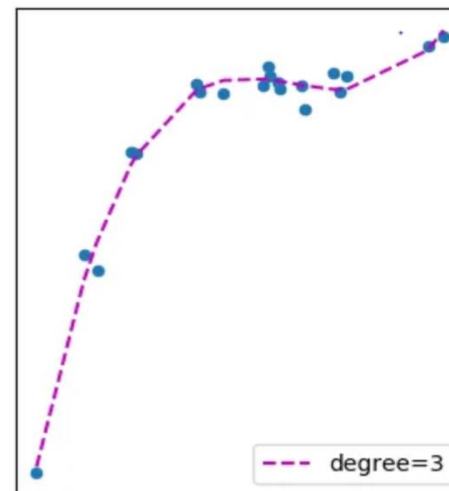
5) Update parameters

$$\theta = \theta - \eta \frac{L'_{\theta}}{N} \quad \eta \text{ is learning rate}$$

Objectives

Non-linear Regression

- ❖ Linear Regression
- ❖ Non-linear Regression
- ❖ Optimal Learning Rate
- ❖ Limitation of Linear Regression



Polynomial Regression

- ❖ Moving Beyond Linearity
- ❖ Polynomial Feature
- ❖ Modeling
- ❖ Degree Choice
- ❖ Non-linear Regression for Fish Dataset



AI VIET NAM

@aivietnam.edu.vn

Thanks!

Any questions?